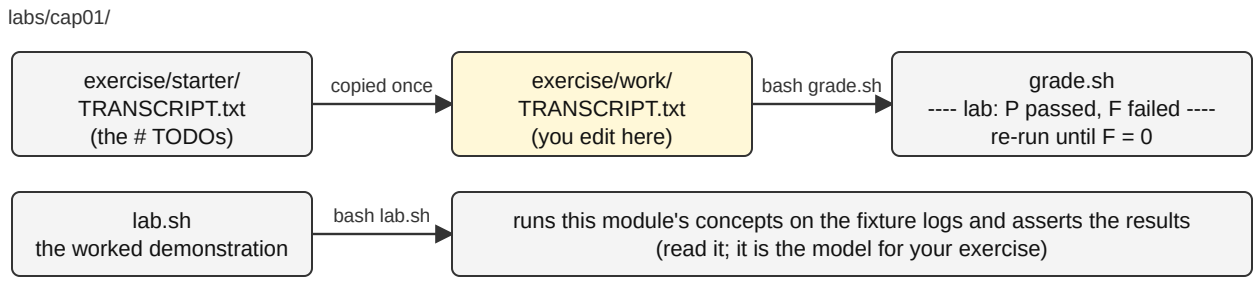
Capstone CAP-01 — Run fraud-clf Live with stackctl, and Hand It Over (with CAP-01R) (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

Everything happens in your own green tree from Lab 9, `labs/lab9-stackctl/exercise/work/`, on a host where `docker info` succeeds (Docker Desktop on macOS; Docker Engine on Ubuntu or inside WSL 2 Ubuntu on Windows 11). Two environments run side by side:

Environment	Project	Published port	Services
`test`	`fraudclf-test`	18080	`gateway` (nginx) → `serve` (fraud-clf FastAPI, built from `app/`)
`prod`	`fraudclf-prod`	18081	the same pair, a separate project, network and volumes

File	What it is	You produce it?
`bin/`, `lib/`, `compose/`, `tests/`, `tools/`	Your finished `stackctl` (LAB-09a/b)	Already done
`app/`	The inherited baseline — **untouched**	Never edited
`envs/test.conf`, `envs/prod.conf`	Environment files; after step 7 they carry the gateway digest	You append one value
`TRANSCRIPT.txt`	The `script` recording of steps 3–7	**Yes**
`MANIFEST.txt`	The unedited output of `tools/record-manifest.sh`	**Yes**
`RUNBOOK.md`	LAB-09c's handover, section 5 now filled from the manifest	**Yes**
`DECISIONS.md`	About 150 words, three answers	**Yes**

Objectives

- **Part 1:** Check the prerequisites and warm up (untimed).
- **Part 2:** Record the live lifecycle of `test`, then `prod`, then the destroy safety.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

- **Part 3:** Tests, manifest and pins.
- **Part 4:** Handover documents and the decisions note.
- **Part 5:** CAP-01R — one broken environment, under observation.

Background / Scenario

This is the summative check for LO-10, LO-11 and LO-12 and the point where Course 0 hands you to the rest of the program. In Lab 9 the mock driver proved your *control logic*; nothing from it counts as evidence here. Now you operate the program's project **for real**: a daemon, both environments, every command in the lifecycle, a prediction through the gateway, the two test suites, and the two handover documents — all recorded in a transcript. A second, shorter part puts one deliberately broken environment in front of you to locate, explain and fix while a trainer watches.

Passing CAP-01 is Course 2's Module 0 ("run the finished thing") done early, and your runbook's *Inherited findings* list is what that course's README tells you to keep.

Required Resources

- LAB-09a/b green (`---- lab9: 34 passed, 0 failed ----`), the LAB-09c runbook without `TODO`, A-10 passed.
- A host with a running Docker daemon and network access for the first image build and pull; `script` (or a terminal recorder).
- The module pages and the Docker documentation; AI assistance **for documentation lookup only** (say so in the note). Not `solution/`, not another learner's tree.

Validation status. The reference `stackctl` passes its mock suite 34/34 and the grader 34/34 in the authoring environment; the live procedure below was **designed** there, not run (no daemon) — verification task V-04. Your transcript is the first evidence either of us has that it works on *your* machine, which is exactly why it is required.

> **Instructor Note:** **Prep (before the first cohort).** Do V-04 yourself: `live-smoke.sh test` and `prod` on the reference, then `record-manifest.sh`; keep the transcripts and the manifest. You need (1) your own `nginx:1.27-alpine` digest *per architecture* to compare learner manifests against — the serve image ID is **not** comparable across machines; (2) a feel for how long the first `up` takes on your network (pip installs scikit-learn/pandas, then the model trains; `WAIT_TIMEOUT=120` may be too short on a slow link — that is why the warm-up is untimed and outside the transcript, and a timeout there is not a learner error if the retry succeeds). Review time ≈ 25 min per learner for CAP-01, ≈ 30 min live for CAP-01R.
>
> **Evidence boundary (decide before the cohort starts).** Without a reviewer who reads the transcript and observes CAP-01R, the result is *self-assessed*. Mock-driver grader output is never CAP-01 evidence; a transcript is. Accept a terminal recording (asciinema, screen capture) in place of `script` output. A transcript edited after the fact is Not met on every criterion it supports — look for prompts and timestamps that do not line up, and for `plan` output whose indentation differs from the tool's.
>
> **Program hand-off.** A pass here is Course 2 Module 0 done early; record it so the Course 2 instructor can start this learner from their *Inherited findings* list.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Part 1: Prerequisites and warm-up (untimed)

Step 1: Confirm the host.

```
cd labs/lab9-stackctl/exercise/work
docker info >/dev/null && echo "daemon ok"
bash ../grade.sh | tail -1
grep -c TODO RUNBOOK.md
```

Expected: `daemon ok`, `---- lab9: 34 passed, 0 failed ----`, `0`.

Step 2: Build once, outside the transcript.

```
bin/stackctl -e test up
CONFIRM_DESTROY=test bin/stackctl -e test destroy
```

The first `up` builds the serve image and pulls the gateway image — several minutes. If `up` times out while pip installs (exit 1), read `bin/stackctl -e test logs` and run `up` again. Nothing from this step goes in the transcript; it is what makes the timed run clean.

Part 2: The recorded lifecycle (timed, ≈35 min)

Step 1: Start the transcript.

Linux and WSL 2: `script -q TRANSCRIPT.txt`. macOS: `script TRANSCRIPT.txt`. Type `exit` at the very end to close it. If `script` is unavailable, copy your terminal into the file by hand — never edit what the commands printed.

Step 2: `test`, in this order.

```
bin/stackctl -e test validate
bin/stackctl -e test plan
bin/stackctl -e test up
bin/stackctl -e test check
bin/stackctl -e test status
bin/stackctl -e test plan            # must say: up would: nothing — converged
bin/stackctl -e test up              # must succeed and change nothing
bin/stackctl -e test logs | tail -5
curl -fsS http://127.0.0.1:18080/api/health; echo " status=$?"
curl -fsS -X POST http://127.0.0.1:18080/api/predict -H 'content-type: application/json'
-d '{"instances": [[120.0, 14, 0.2, 1]]}'; echo
```

Expected (designed; confirm on your machine): `validate → ok: test → project fraudclf-test on port 18080 (…)` plus the `warning:` about the tag-only gateway pin and the `note:` about the local serve image; `plan → up would: create project fraudclf-test (2 services)`; `up → up: done — 2 of 2 services running` (fast — the warm-up built the image); `check → check: http://127.0.0.1:18080/api/health → [{"status":"ok"}] ok`; `status → gateway` and `serve`; the second `plan → up would: nothing — converged`; the second `up → done — 2 of 2` with Compose reporting the containers as `Running`, not `Recreated`; `curl` health `→ {"status":"ok"} status=0`; predict `→ {"probabilities":[[p_legit,p_fraud]]}` — any two numbers summing to 1 are a correct answer from the baseline model (the exact values depend on training data regenerated at build and are not graded).

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 3: `prod`, the same sequence with `-e prod` — while `test` is still up.

Then show the two are isolated:

```
docker compose ls
curl -fsS http://127.0.0.1:18081/api/health; echo " status=$?"
```

Expected: both `fraudclf-test` and `fraudclf-prod` listed as `running(2)`; `{"status":"ok"}` `status=0` on 18081.

Step 4: Destroy safety, in the transcript.

```
bin/stackctl -e test destroy; echo "status=$?"
CONFIRM_DESTROY=prod bin/stackctl -e test destroy; echo "status=$?"
CONFIRM_DESTROY=test bin/stackctl -e test destroy; echo "status=$?"
docker compose ls
CONFIRM_DESTROY=prod bin/stackctl -e prod destroy; echo "status=$?"
```

Expected: `destroy refused: set CONFIRM_DESTROY=test …` and `status=2`, twice; `destroy: project fraudclf-test removed (containers, network, volumes)` and `status=0`; `docker compose ls` still listing `fraudclf-prod` **running**; then `status=0` for `prod`.

Symptom	Cause	Fix
`up` fails mentioning `address already in use`	Another app holds 18080/18081	Change `PORT` in the conf, `validate`, `plan`, `up`
`up` exits 1 after `WAIT_TIMEOUT`	Model still training at first build, or a slow pull	Read `logs`; retry; the warm-up should have prevented this
Second `plan` reports drift	A render-path difference you introduced (S09.4)	Explain it in D1; fix the render so both renders compare equal
`check` exits 1 right after `up`	Gateway up, path wrong, or port drift	`validate` shows `health GET <path>`; compare with what the app serves

Part 3: Tests, manifest and pins (≈15 min)

Step 1: Both suites, in the transcript.

```
bash tests/run.sh | tail -1
bash tests/live-smoke.sh test
```

Expected: `# 34 passed, 0 failed`; the smoke test builds (cached), brings up, checks, predicts once, proves convergence, destroys, and ends `== live smoke OK ==`.

Step 2: Manifest and pins.

```
bash tools/record-manifest.sh | tee MANIFEST.txt
```

Expected: per environment, one `IMAGE_GATEWAY … → nginx@sha256:…` line and one `IMAGE_SERVE fraudclf-serve:m0 → sha256:… (local build of APP_DIR/Dockerfile.course)` line. Append `@sha256:<digest>` **from that output** to `IMAGE_GATEWAY` in both conf files, then:

```
bin/stackctl -e test validate
bin/stackctl -e prod validate
exit            # closes the transcript
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Expected: the tag-only `warning:` is gone; the `note:` about the locally built serve image stays (it has no digest to pin — it was never pushed; its ID exists only on the machine that built it).

Never type a digest or an image ID from a document, a chat answer or this manual. A copied value is a claim without an observation; the course treats it as fabricated evidence even when the string happens to be correct.

Part 4: Handover and the decisions note (≈10 min)

Step 1: `RUNBOOK.md`.

Complete from LAB-09c, with section 5 now filled from `MANIFEST.txt` and section 6 listing the inherited findings. `app/` stays untouched: the planted example keys in `app/config.py` are an inherited finding that belongs in section 6, not a fix you make here.

Step 2: `DECISIONS.md` — about 150 words in total, your own words.

D1. Your transcript shows `plan` saying `converged` and a second `up` that changed nothing. What in *your code* makes that true, and what in Compose? What change would `plan` *not* have seen?

> **Answer:** **Met:** in the code, `up` renders the same files from the same conf and `plan` compares a fresh render with `.render/test/` byte for byte (`diff -q` on both files) and counts running services, so it prints `nothing — converged`; in Compose, `up` compares each service's definition with the running container's and leaves unchanged services alone. What `plan` cannot see: a change inside `app/` — the image name `fraudclf-serve:m0` is unchanged, so the render is identical while the image that would be built is not. **Strong:** adds that Compose prints `Running` for untouched services, that the bind-mount and build-context paths are deliberately rendered as stable paths so the two renders are identical, or that the manifest records the image *ID* precisely because the tag cannot carry that information.
>
> *Calibration.* Strong: "My `plan` diffs the fresh render against `.render/test/stack.yml` and `gateway.conf` — identical because the mount path and build context are rendered as fixed paths either way — and `compose ps` says 2 running, so it prints converged. Compose's own `up` showed `Container fraudclf-test-serve-1 Running` the second time. If I edited `app/serve.py` nothing would change in the render, so `plan` would still say converged — that's why the manifest keeps the image ID." Not met: "Because Compose is idempotent."

D2. `destroy` refused twice before it worked. Name the two conditions it checks and say what each one protects against.

> **Answer:** **Met:** `CONFIRM_DESTROY` must equal the selected environment's name (protects against destroying the wrong environment, or a generic `yes` propagating), and the lock must be free (protects against destroying while another operator's `up` is mid-flight). **Strong:** adds that `down -v` is scoped by `--project-name`, so even a confirmed destroy cannot touch `prod`; that there is no prune for the same reason; or that Course 2's promotion approval is the same gate with a human in it.

D3. Where did the gateway digest and the serve image ID in `MANIFEST.txt` come from, and why would copying either from the course page or a chat answer be a critical failure even if the string happened to be correct?

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> **Answer: Met:** the gateway digest from `docker image inspect --format '{{index .RepoDigests 0}}'` and the serve image ID from `--format '{{.Id}}'`, both run by `tools/record-manifest.sh` on the machine that pulled and built the images; a copied value is a claim without an observation — the reviewer cannot tell, so the course treats it as fabricated evidence, the same "observed vs designed" rule used throughout. **Strong:** adds that a digest is per platform/architecture, that a tag can move while a digest cannot, and that the serve image has no digest anywhere because it was never pushed.

Rubric R-11 and hand-in

Deliver the `work/` tree (`bin/`, `lib/`, `envs/*.conf` with the recorded gateway digest, `app/` untouched, `tests/`, `RUNBOOK.md`, `MANIFEST.txt`), `TRANSCRIPT.txt` and `DECISIONS.md`. Levels **Not met** (0) · **Met** (1) · **Strong** (2), scored `weight × level / 2`. **Pass** = total ≥ 80 % of 26 **and** no *critical* criterion at Not met. CAP-01 may be re-attempted after remediation with a fresh transcript.

ID	Criterion	Evidence	Weight	Critical
R-11.1	Live operation of `test` then `prod`: `up` healthy, `check` ok, a prediction answered, `status`, both projects listed and isolated	transcript, Part 2	4	—
R-11.2	Health contract exact: `check` 0 live; mismatch and unreachable distinguished (3 vs 1)	transcript; mock suite	2	—
R-11.3	Validation precedes every mutation and `plan` never mutates	code + transcript + mock suite	3	**yes**
R-11.4	Idempotency: second `up` succeeds and `plan` reports converged	transcript	2	**yes**
R-11.5	Destroy safety: refuses without `CONFIRM_DESTROY=<this env>`; `down -v` scoped to the project; no global prune anywhere	transcript; code	3	**yes**
R-11.6	Tests: `tests/run.sh` 34/34 and `live-smoke.sh` OK, both in the transcript	transcript	3	—
R-11.7	Handover: `RUNBOOK.md` complete incl. inherited findings; `MANIFEST.txt` recorded on the operating machine; gateway pinned; `app/` untouched; no real secrets anywhere	files + transcript	3	**yes** (fabricated value, modified `app/`, or a real secret)
R-11.8	Decisions note: three answers that name the mechanism in your own words	`DECISIONS.md`	6	—

A flawless transcript with no decisions note scores 20/26 = 77 % — **not a pass**.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Level descriptors.

- **R-11.1 (4).** *Strong:* the full `test` then `prod` sequence in order, both `ups` healthy, both `checks` ok, a predict answered on both, `docker compose ls` listing both, `curl` on both ports. *Met:* both brought up and checked, but out of order, or `status`/`ls`/predict missing, or one environment brought up only after the other was destroyed (isolation not demonstrated). *Not met:* one environment never reached `check` ok live.
- **R-11.2 (2).** *Strong:* live `check` 0 **and** the mock suite's four `check` cases pass, and `cmd_check` uses a byte-exact capture and `HEALTH_PATH`. *Met:* live check 0 but the capture is a plain `$( )` (the mock suite shows `body + trailing newline → 3` failing). *Not met:* `check` compares with `grep`/substring, hard-codes `/api/health`, or does not distinguish 1 from 3.
- **R-11.3 (3, critical).** *Strong:* the dispatcher validates before every command, `plan` renders only to `$WORK`, the mock checks pass, and D1/D2 show the learner knows why. *Met:* passes the checks but `plan` writes into `.render/` (read `cmd_plan` — the grader cannot see this) or `up` validates only when called directly. *Not met:* any `up`/`down` reachable before validation, or a `plan` that calls `up`.
- **R-11.4 (2, critical).** *Strong:* second `plan` says converged, second `up` is in the transcript and recreated nothing (Compose prints `Running`), and D1 names the change `plan` cannot see. *Met:* second `up` succeeded but `plan` reported drift because of a self-inflicted render-path difference the learner explains. *Not met:* second `up` recreated services or failed, or the step is missing.
- **R-11.5 (3, critical).** *Strong:* two refusals and one success in the transcript, `docker compose ls` proving `prod` survived, `down -v` scoped by `--project-name`, `grep -r prune` over the tree empty. *Met:* refusals correct but no proof the other environment survived. *Not met:* any accepted confirmation other than the exact name, or any `docker system prune`/`volume prune` in the tree or transcript.
- **R-11.6 (3).** *Strong:* `# 34 passed, 0 failed` and `== live smoke OK ==` both in the transcript, unmodified `tests/`. *Met:* both pass but `tests/run.sh` was edited (`diff -r tests ../starter/tests`; ask why). *Not met:* either suite missing or failing.
- **R-11.7 (3, critical for fabricated values / modified app / real secrets).** *Strong:* runbook at the LAB-09c Met bar or better including section 6, `MANIFEST.txt` unedited, confs carry the `@sha256:` pin matching the manifest, re-run `validate` shows no tag-only warning, `diff -r app ../starter/app` empty. *Met:* manifest present but confs still tag-only, or a thin section 6. *Not met (critical):* a digest or ID not in the learner's own manifest/transcript, or a gateway digest not matching your V-04 digest for the same architecture without an explanation (a different architecture is legitimate — ask); a modified `app/`; any *real* token, password or key in any file or transcript. The planted `AKIAIOSFODNN7EXAMPLE` pair in `app/config.py` is AWS's documented example and is **not** a finding against the learner.
- **R-11.8 (6).** *Strong:* all three answers name the mechanism in the learner's own words with a reference to their own transcript. *Met:* two of three at mechanism level, or three correct but generic answers. *Not met:* fewer than two mechanism-level answers, or text that reads as pasted documentation.

Scoring procedure (≈25 min). 1) Read `TRANSCRIPT.txt` top to bottom once, ticking the steps. 2) `cd work && bash tests/run.sh | tail -1 && grep -rn prune bin lib tests; diff -r`

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> `tests ../starter/tests; diff -r app ../starter/app` (expect no prune, no diffs). 3) Open `envs/*.conf` and `MANIFEST.txt`; compare the gateway digest with the manifest and with your V-04 digest; check the serve ID is 64 hex digits and appears in the live transcript's build. 4) Level each criterion; `Σ weight × level / 2`; apply the critical gates; pass ≥ 20.8 → **21/26**. 5) Feedback per criterion below Strong.
>
> **Worked arithmetic.** All Strong = 26/26. Everything Strong but no note: 20/26 = 76.9 % → fail. Everything Strong, note Met: 23/26 = 88 % → pass. R-11.1 Met, R-11.7 Met, rest Strong: 22.5/26 = 86.5 % → pass with feedback. All Met: 13/26 → fail. Any critical Not met → fail regardless (24/26 with a pasted digest, or with `config.py` "fixed" → fail).
>
> **Things that legitimately differ between machines:** the gateway digest per architecture; the serve image ID on every machine and rebuild; the Compose version line; `up` duration; the predict probabilities; on a Mac without GNU tools, the `sed -i ''` form and a lock that cannot be taken without Homebrew `flock` — accept a documented container run of the lock-dependent commands only in that case; on Windows 11 everything runs inside WSL 2, so the transcript looks like Ubuntu's.
>
> **Remediation map.** R-11.3/11.4 → S09.3–S09.4 and LAB-09b; R-11.5 → S09.5 and R2 of LAB-09b; R-11.2 → S09.6; R-11.7 → S09.7 and LAB-09c (and `COURSE0-NOTE.md` for a modified `app/`); R-11.6 → M08 then LAB-09b; R-11.8 → the module's "Why" paragraphs, then rewrite the note from the transcript. **Re-attempt:** same rubric; a fresh transcript is required.
>
> **Evidence handling.** Keep `TRANSCRIPT.txt`, `MANIFEST.txt`, `DECISIONS.md`, the two conf files, the `diff -r app` result and the rubric sheet. Strip nothing from transcripts (the planted example keys may appear if a learner `cats config.py`; that is expected).

Part 5: CAP-01R — under observation (30 min)

Your trainer hands you a copy of your own tree with **one** environment altered: ports `28080`/`28081`, projects `fraudclf-r-test`/`fraudclf-r-prod`, and one injected fault somewhere in `envs/`, `compose/` or the host state. With the trainer watching (live or on a recording):

1. Run the lifecycle until something fails.
2. **Say what you think the fault is before you fix it**, naming the exit code and the line of output that told you.
3. Fix it and verify with `check → [{"status":"ok"}] ok`.
4. Add the symptom and recovery to your `RUNBOOK.md` if it is not there yet.

Rubric R-12. *Critical:* the fault is **located and explained** from the tool's own output before the fix (R-12.1, weight 4); the fix is **verified by `check`** (R-12.2, weight 3). Non-critical: the runbook gains the case or already covered it (R-12.3, weight 2). Pass = ≥ 80 % and no critical at Not met. Fixing by trial and error without naming the cause fails R-12.1 even if the stack ends healthy; "fixing" `app/` to match a broken configuration is Not met.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Preparing the copy. `cp -r work cap01r && cd cap01r && rm -rf .render envs/*.lock`, then `sed -i 's/^PORT=18080/PORT=28080/; s/^PROJECT=fraudclf-test/PROJECT=fraudclf-r-test/' envs/test.conf` and `sed -i 's/^PORT=18081/PORT=28081/; s/^PROJECT=fraudclf-prod/PROJECT=fraudclf-r-prod/' envs/prod.conf` (BSD `sed -i ''` on a Mac without GNU tools). Inject **one** fault; tell the learner only "one environment is altered". Faults marked *observed* were reproduced with the reference and the mock driver; the rest are *designed* and need a daemon — run them yourself once before using them.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Tier	ID	Injection (in `cap01r/`)	Symptom the learner must read	Diagnosis and fix	Status
A	F1	`sed -i 's/^PORT=28080/PORT=280a80/' envs/test.conf`	`PORT must be a number, got: 280a80`, exit **2**, no `up` call	non-numeric port; fix the line, `validate` → 0	observed
A	F2	`sed -i '/^HEALTH_BODY/d' envs/test.conf`	`missing required key: HEALTH_BODY`, exit 2	restore `HEALTH_BODY={"status":"ok"}`	observed
A	F3	`sed -i 's/^HEALTH_BODY=.*/HEALTH_BODY={"status": "ok"}/' envs/test.conf` (a space)	`check: … returned [{"status":"ok"}], expected [{"status": "ok"}]`, exit **3**; `plan` shows the tell	the *expected* contract was edited; fix the conf, `check` → 0	observed (mock); live designed
A	F8	`sed -i 's|^HEALTH_PATH=.*|HEALTH_PATH=/health|' envs/test.conf`	`validate` prints `health GET /health`; live: app answers 404 → `unreachable`, exit **1** — not 3	the path, not the body; fix the conf, `check` → 0	tell observed; live designed
B	F4	`sed -i 's/^PROJECT=fraudclf-r-prod/PROJECT=fraudclf-r-test/' envs/prod.conf`	`validate -e prod` → `ok: prod → project fraudclf-r-test …`; live: `up -e prod` recreates `test`'s project, `check -e test` → unreachable, exit 1, one project in `compose ls`	both environments share a project; fix `PROJECT`, `up -e test`, `up -e prod`, `check` both	tell observed; live designed
C	F5	`sed -i 's/serve:8080/serve:8081/' compose/gateway.conf.tmpl`	`validate` → 0; `plan` shows drift in `proxy_pass`; live: `--wait` times out, `up` exit 1; `logs` shows nginx `connect() failed … upstream`	wrong upstream port; fix the template, `up`, `check`	drift observed; live designed
C	F6	trainer runs `( exec 9>envs/test.lock; flock 9; sleep 600 ) &`	`test is locked by another stackctl run (…)`, exit **1** immediately; `fuser envs/test.lock` shows a PID	another holder; identify, confirm it is not a real operator, end it, re-run	observed
C	F7	`sed -i 's/^IMAGE_GATEWAY=nginx:1.27-alpine$/IMAGE_GATEWAY=nginx:1.27-alpnie/'`	`validate` → 0; `up` → pull error (`manifest … not found`), exit 1	tag typo; fix the conf, `up`	designed

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Tier	ID	Injection (in `cap01r/`)	Symptom the learner must read	Diagnosis and fix	Status
		`envs/test.conf` (only if not digest-pinned; needs network)			
C	F9	`mv app/Dockerfile.course app/Dockerfile.course.bak`	`APP_DIR app/ has no Dockerfile.course …`, exit **2** from every command	restore the file; note nothing was mutated because validation refused first	observed

Equivalence rule. First attempt and reassessment use different faults from the same tier (A: F1/F2/F3/F8/F9 — F3 and F8 are the instructive pair, body vs path; B: F4 alone — reassess with F5; C: F5/F6/F7). Tier A for entry-level cohorts; tier C where learners have a daemon and network; never F7 offline.

Observation protocol (≈30 min). 1) Say only: "one environment in this copy is altered; find it, tell me what it is before you fix it, fix it, prove it with `check`." 2) Start timing at the first command. 3) Do not answer questions about the fault; you may answer questions about the tool's documented behaviour — note that you did. 4) Record: the exact output line the learner cited, the sentence in which they named the cause, whether the fix was targeted (one change) or exploratory, and the final `check` line. 5) Stop at 30 min; an unfinished attempt is scored on what was said and shown.

R-12 descriptors. R-12.1 Located and explained (4, critical). *Strong:* names the fault, the exit code and the output line *before* editing anything; for F4 also names the broken invariant (two environments, one project); for F3 vs F8 says which side — contract or route — is wrong. *Met:* names it after one wrong guess that the tool's own output immediately refuted. *Not met:* trial and error, or names the fault only after the stack is healthy, or "fixes" `app/`. **R-12.2 Fixed and verified (3, critical).** *Strong:* one targeted change, then `validate`/`up`/`check` as appropriate, ending in `check: … → [{"status":"ok"}] ok`; for F4 verifies both environments. *Met:* extra harmless changes, verified with `check`. *Not met:* `check` never run, or "it works" asserted from `up` alone. **R-12.3 Runbook (2).** *Strong:* the case was already there or is added with symptom line, code and fix. *Met:* added in prose without the exit code. *Not met:* not added. Pass = ≥ 7.2 → **8/9** with both critical ≥ Met.

Calibration (F8). *Strong:* "`check` exits 1, not 3, and curl said 404 — so nothing is wrong with the body, the path is wrong: `validate` prints `health GET /health` and the app only has `/api/health`. The fault is `HEALTH_PATH` in `test.conf`." *Not met:* the learner edits `app/src/serve.py` to add a `/health` route so the check passes.

Course close. Record CAP-01 (score, critical gates, evidence location) and CAP-01R (fault ID, time, R-12 levels). Completion wording: "self-assessed" unless a reviewer scored R-11 and observed R-12. Pass the learner's *Inherited findings* list to the Course 2 instructor. Maintenance triggers: Compose `--wait` semantics, the nginx tag, `python:3.12-slim` and the baseline's `requirements.txt` (a pip resolution change can break the build — pin in `Dockerfile.course` if it does), and any change to the baseline's `/api/health` response.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.